

CPEN 416 / EECE 5710 Lecture 11

# Quantum compilation II

Thursday 16 October 2025

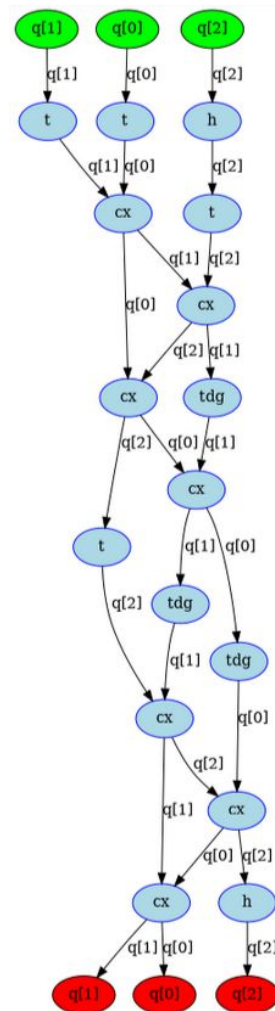
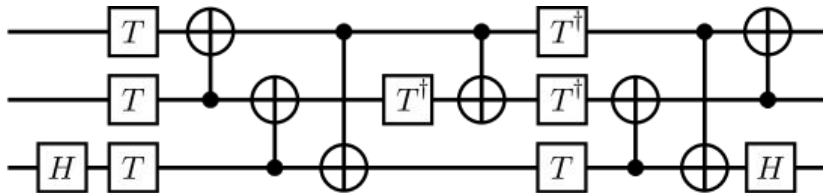
# Announcements

- Final project group + paper selection **due 17:00 today**
  - After the deadline, I will make groups in PrairieLearn and create the weekly surveys
- Monday's tutorial: **Tutorial Assignment 2**
- Quiz 5 on Tuesday (about this week)

Last time

## Circuit metrics

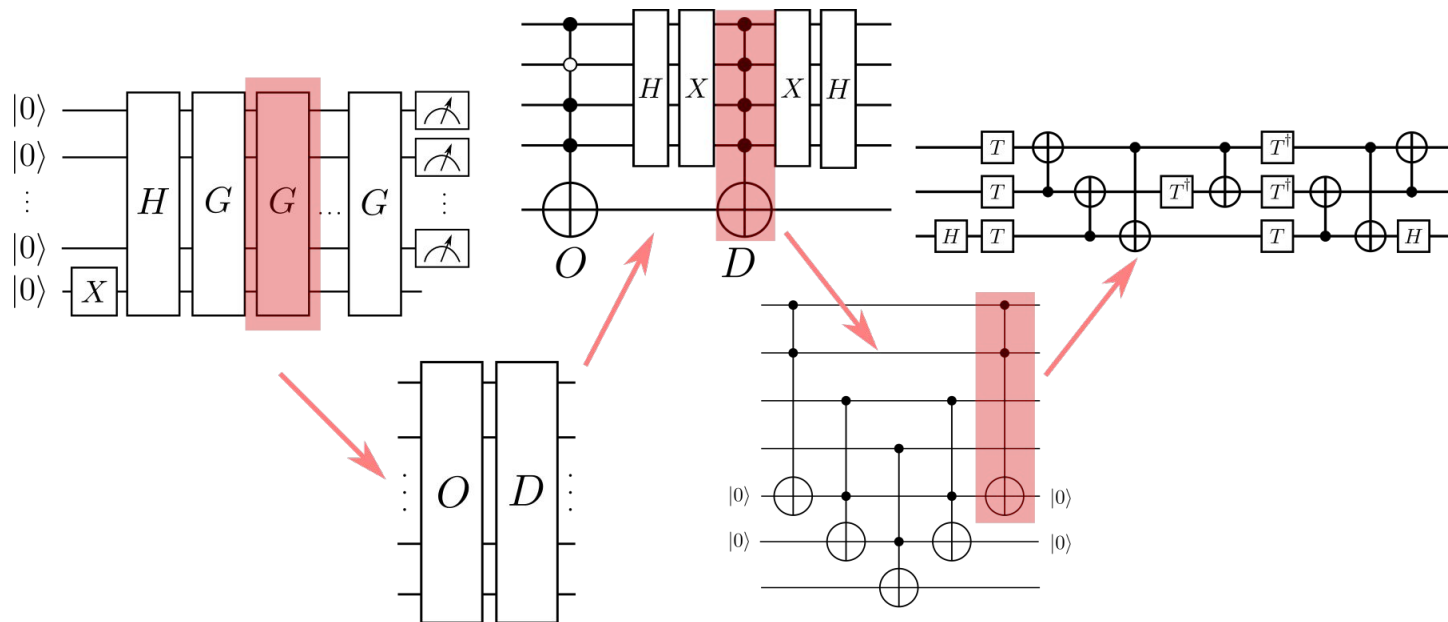
- width (number of qubits)
- gate count (1 or 2-qubit gates, or specific gates)
- depth (of whole circuit, or specific gate)
  - longest path in directed, acyclic graph representation



Last time

## We opened the black box

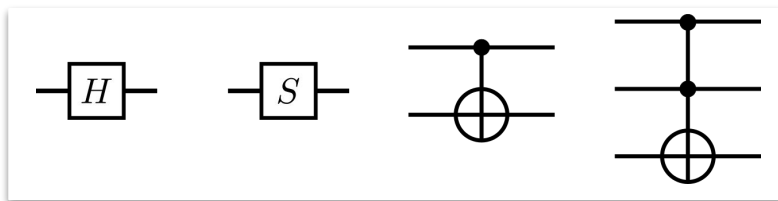
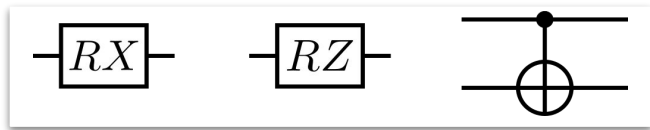
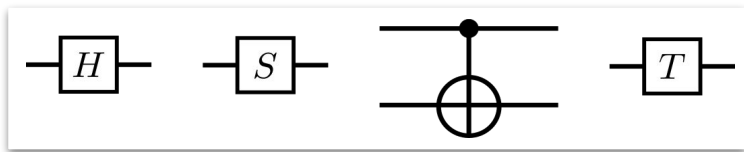
High-level gates in quantum circuits must be synthesized or decomposed into gates from a **universal gate set**.



Last time

# Universal gate sets

With just a few gates, we can implement any unitary either exactly, or approximately up to arbitrary precision.



Last time

## Exercise

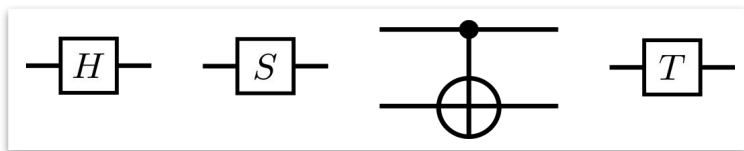
Express Hadamard in terms of  $\{RX, RY, RZ\}$

*Hint:* there are many ways to do this!

Last time

## Exercise

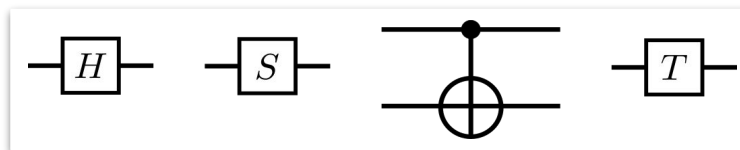
Express CZ using gates from



Last time

## Take-home exercise

Express controlled Hadamard using gates from





# Today

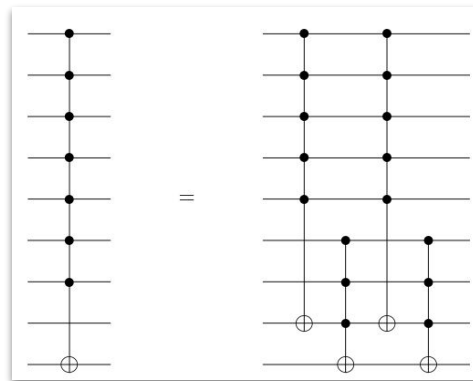
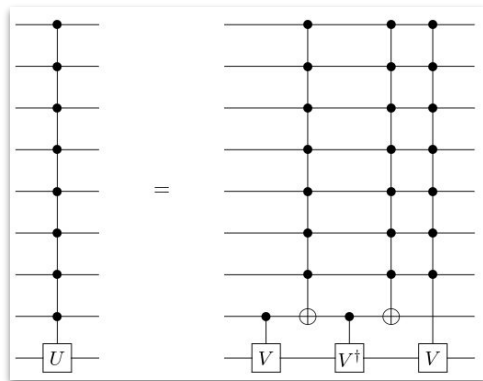
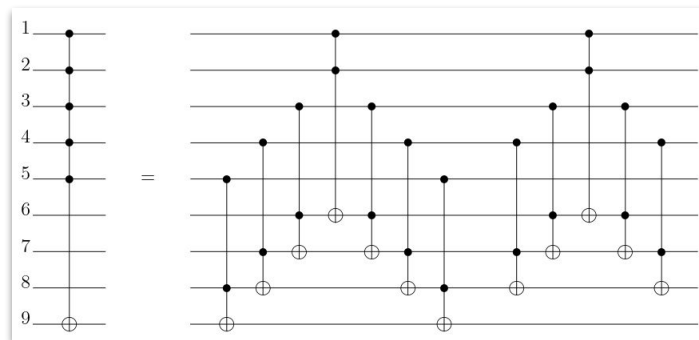
- decompose multi-controlled operations into 1- and 2-qubit gates
- describe the different parts of the quantum compilation stack
- compile and optimize Grover circuits to a topology-restricted quantum processor

L01: circuit decomposition

## Example

Let's decompose a multi-controlled NOT.

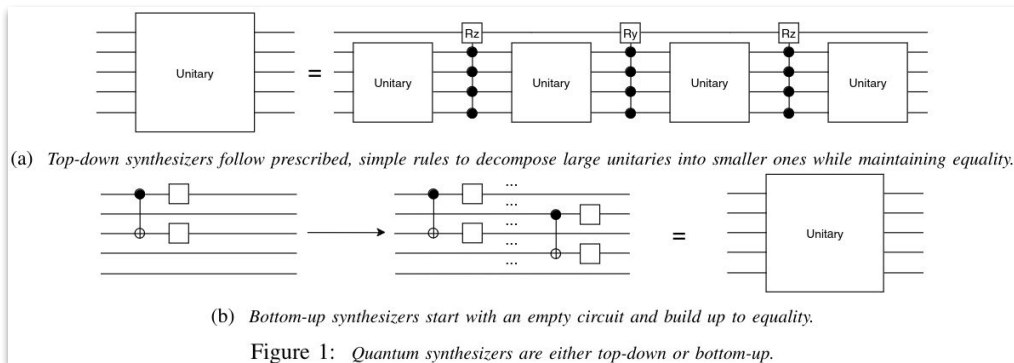
# Multi-controlled NOT decomposition strategies



# Synthesis and decomposition techniques

For under-specified or arbitrary operations, or for special gate sets, we may need to *find* a decomposition.

- search problem
- number-theoretic techniques
- numerical methods



# Challenges

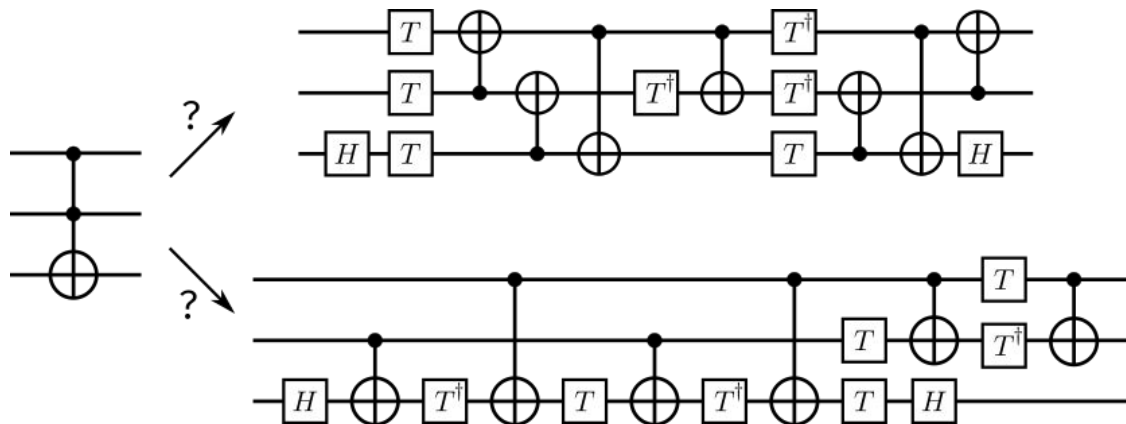
Synthesis is hard!

- algorithm complexity and computational resources

# Challenges

Synthesis is hard!

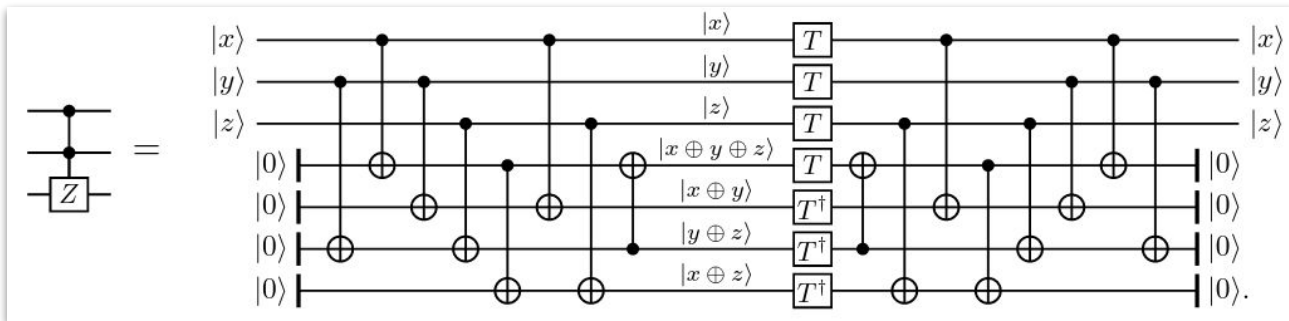
- algorithm complexity and computational resources
- how to choose the best decomposition to *enable further optimization* down the line?



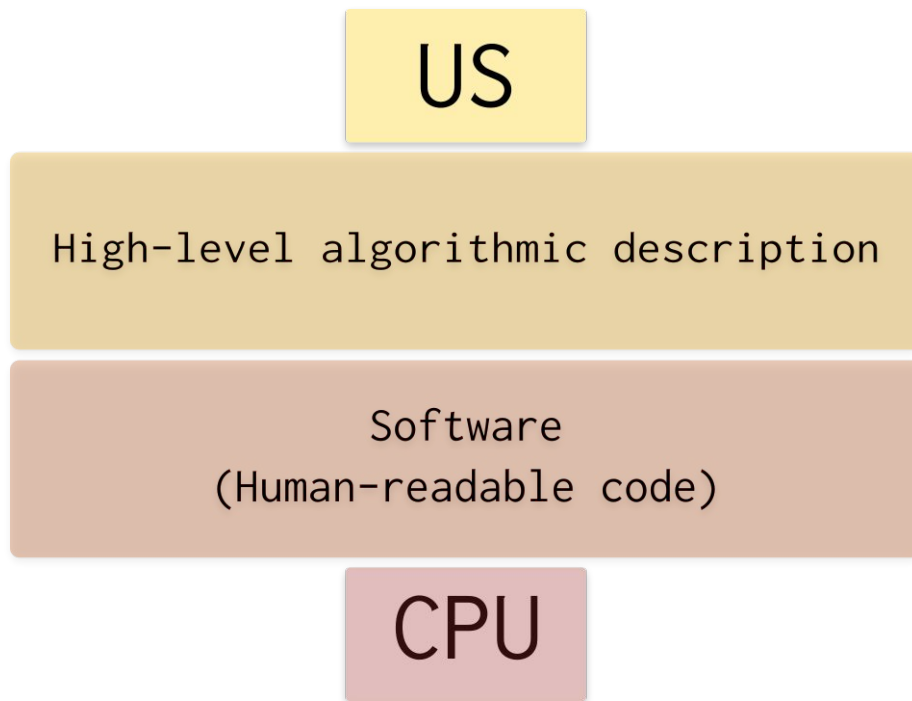
# Challenges

Synthesis is hard!

- algorithm complexity and computational resources
- how to choose the best decomposition to *enable further optimization* down the line?
- manage tradeoffs between various resources



## L02: the quantum compilation stack





# Compilers are essential

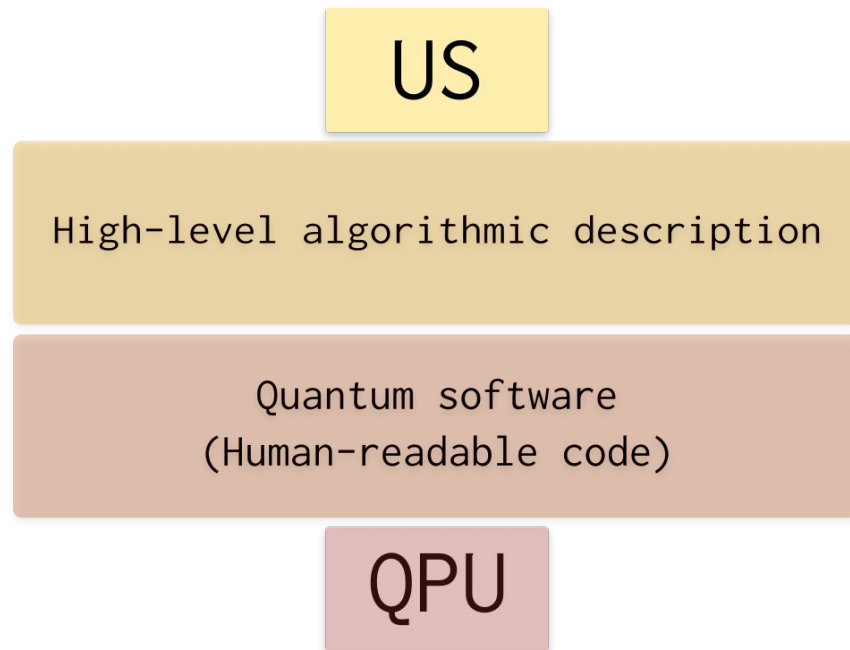
```
#include<stdio.h>

void main() {
    printf("Hello, world!");
}
```

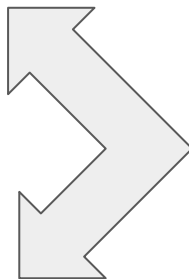
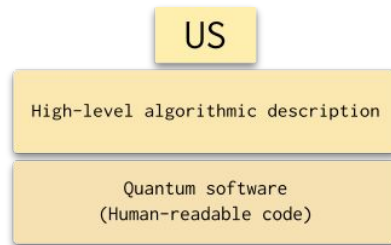
```
.file "hello_world.c"
.text
.section .rodata
.LC0:
.string "Hello, world!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rdi
movl $0, %eax
call printf@PLT
nop
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 9.2.1-9ubuntu2) 9"
```

```
11a0 ff48c1fd 03741f31 db0f1f80 00000000 .H...t.1.....
11b0 4c89f24c 89ee4489 e741ff14 df4883c3 L...L..D..A...H..
11c0 014839dd 75ea4883 c4085b5d 415c415d .H9.u.H...[A\A]
11d0 415e415f c366662e 0f1f8400 00000000 A^A_.ff.....
11e0 f30f1efa c3
Contents of section .fini:
11e8 f30f1efa 4883ec08 4883c408 c3 ....H...H....
Contents of section .rodata:
2000 01000200 48656c6c 6f2c2077 6f726c64 ....Hello, world
2010 2100
Contents of section .eh_frame_hdr:
2014 011b033b 40000000 07000000 0cf0ffff ...;@.....
2024 74000000 2cf0ffff 9c000000 3cf0ffff t...,.....<...
2034 b4000000 4cf0ffff 5c000000 35f1ffff ....L...\...5...
2044 cc000000 5cf1ffff ec000000 ccf1ffff ....\.....
2054 34010000
Contents of section .eh_frame:
2058 14000000 00000000 017a5200 01781001 .....zR...x...
```

## In an ideal world...



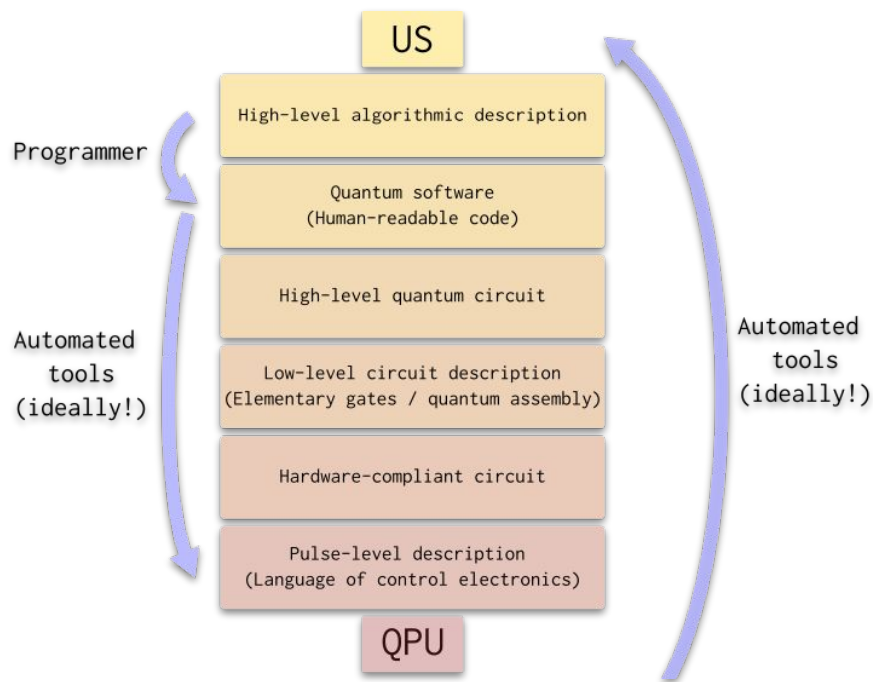
# This lecture



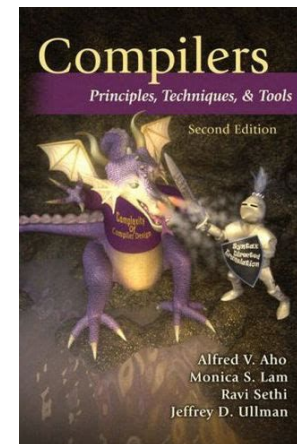
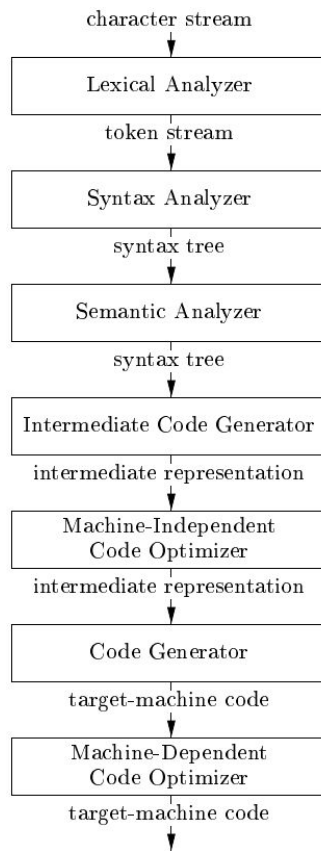
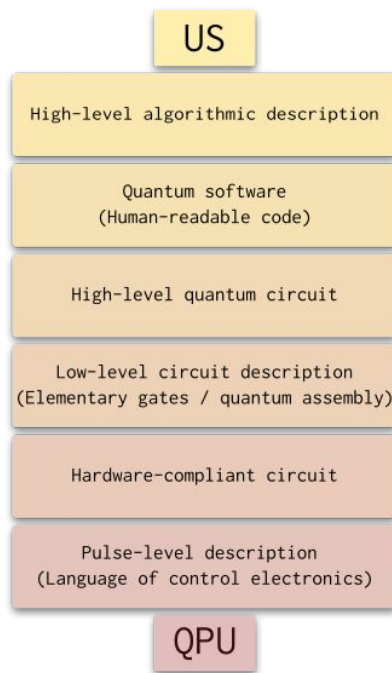
What happens in between?

QPU

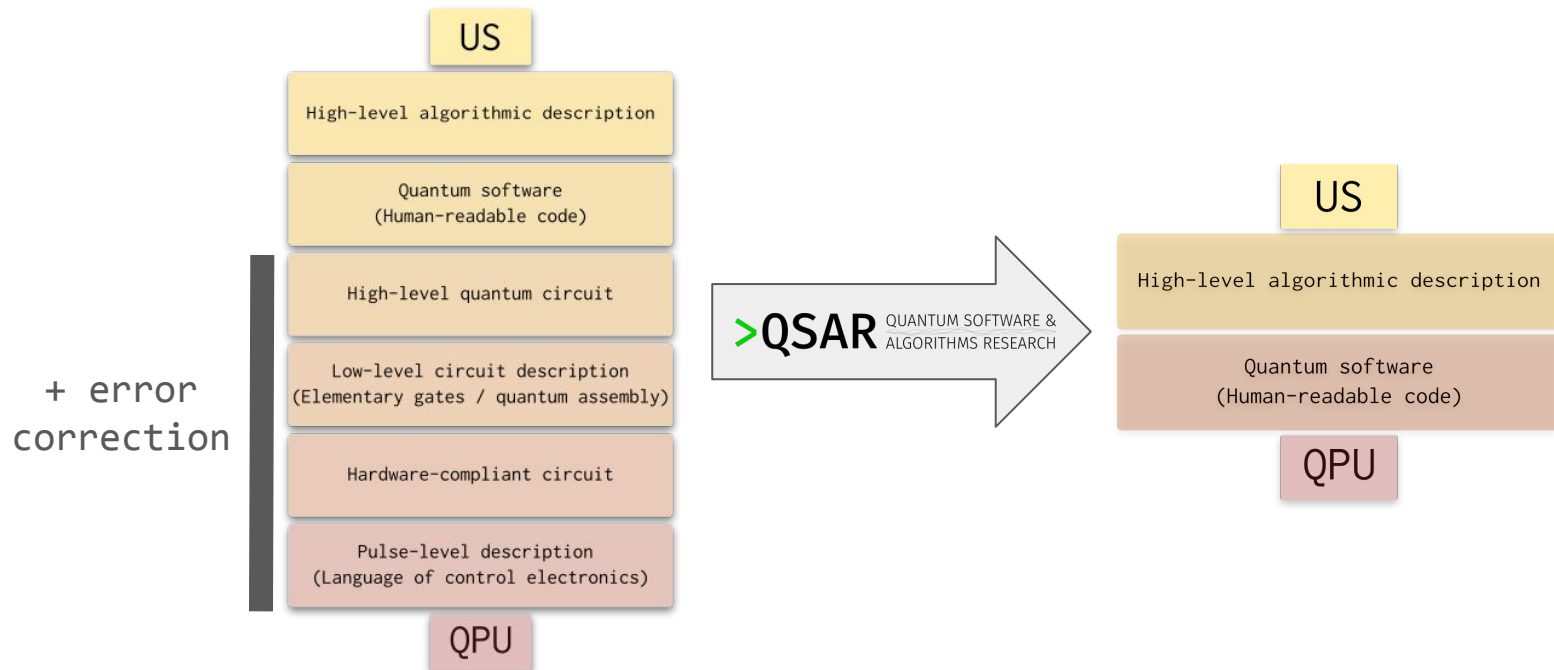
# The quantum compilation stack



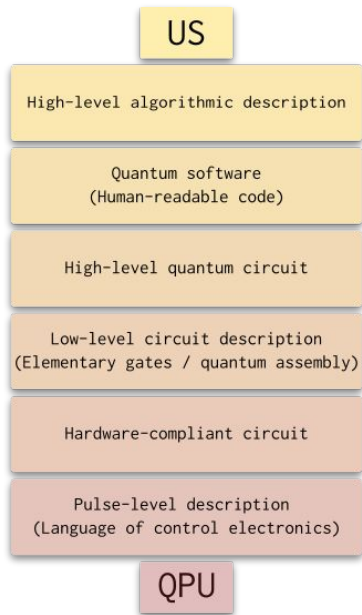
# The quantum compilation stack



# Goal of my research group



# Compilation



Implemented as a sequence of modular *passes*, or *transforms*, that modify a circuit in various ways.

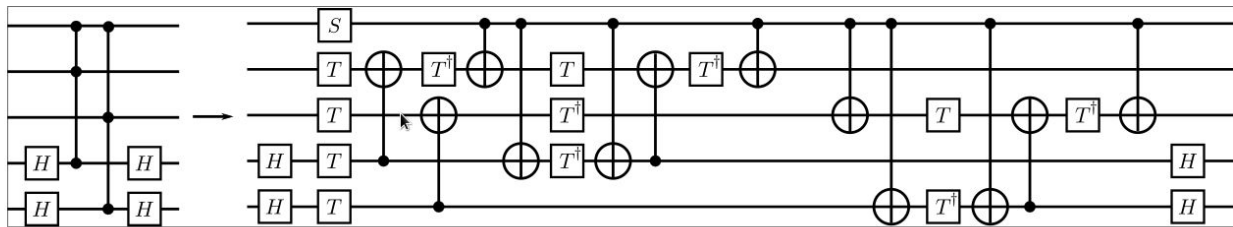
Loosely divided into 3 stages:

- decomposition and synthesis
- hardware-independent optimization
- hardware-dependent optimization

## Circuit optimization

Machine-independent (or dependent) sequence of *passes* to reduce:

- gate count (overall, or for particular gate)
- circuit depth
- qubit count





## Optimization passes

Passes have varying levels of complexity:

- inverse cancellation

## Optimization passes

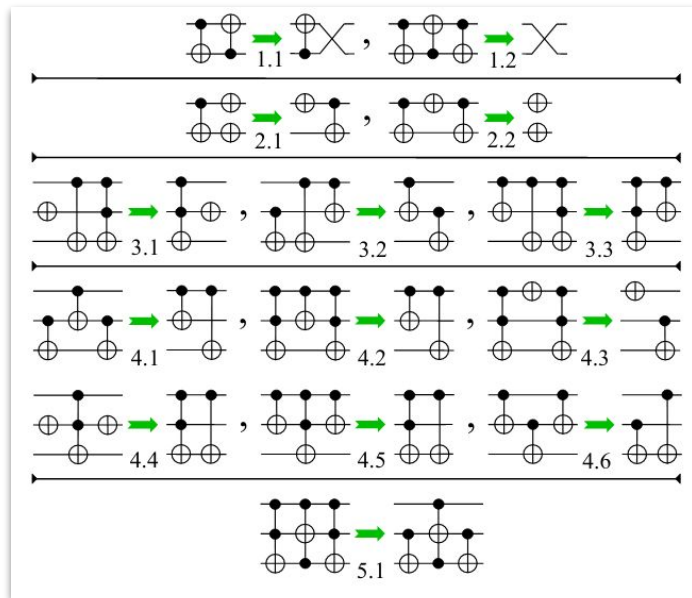
Passes have varying levels of complexity:

- inverse cancellation
- rotation merging

## Optimization passes

Passes have varying levels of complexity:

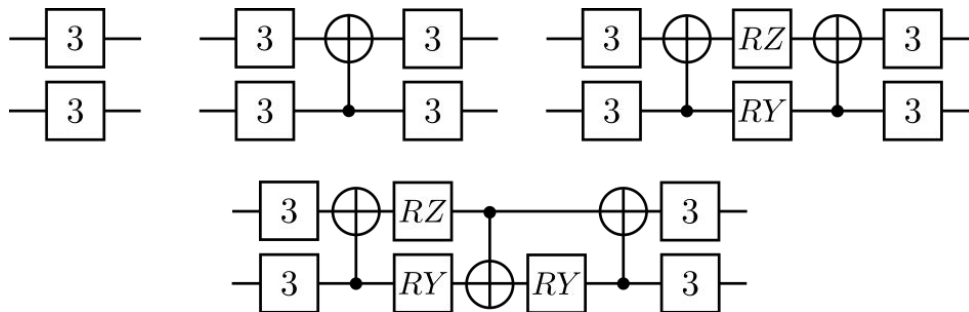
- inverse cancellation
- rotation merging
- template matching



## Optimization passes

Passes have varying levels of complexity:

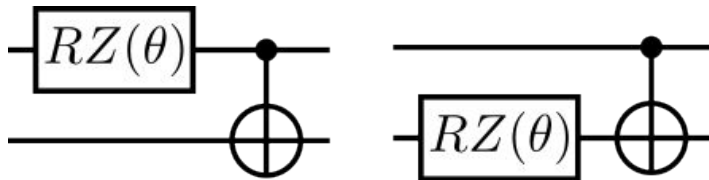
- inverse cancellation
- rotation merging
- template matching
- two-qubit block *resynthesis*



## Commutation

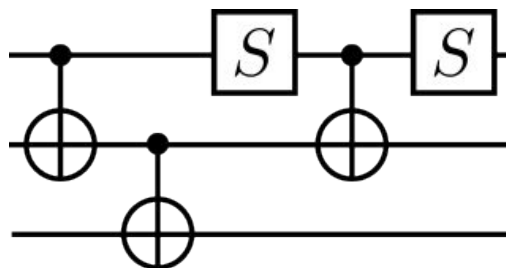
Commutation relations can help us move gates “through” each other to enable optimizations.

**Exercise:** do the following gates commute?

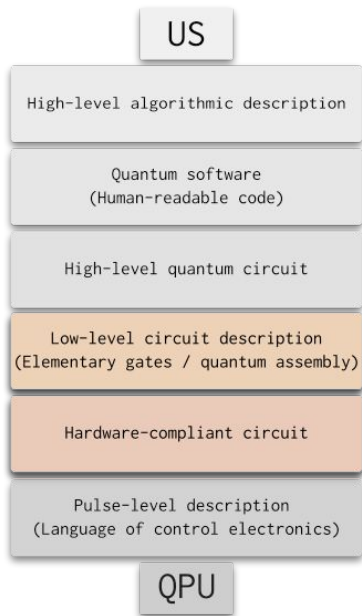


## Exercise

Determine this circuit's resource counts, then optimize it.  
What is the minimum depth?

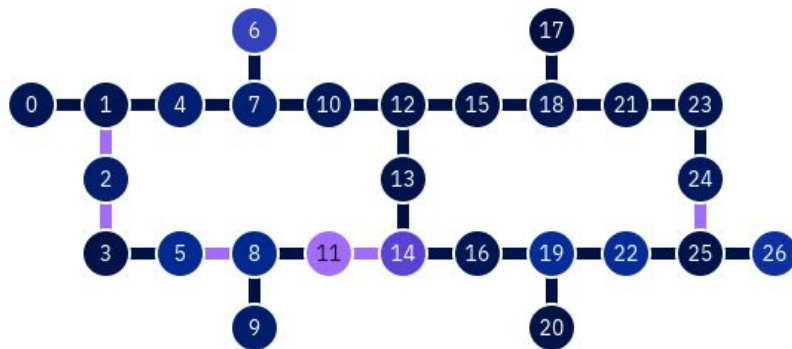


# Qubit placement and routing



Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology



IBM Q Kolkata – screencap 2023-08-24

# Qubit placement and routing

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

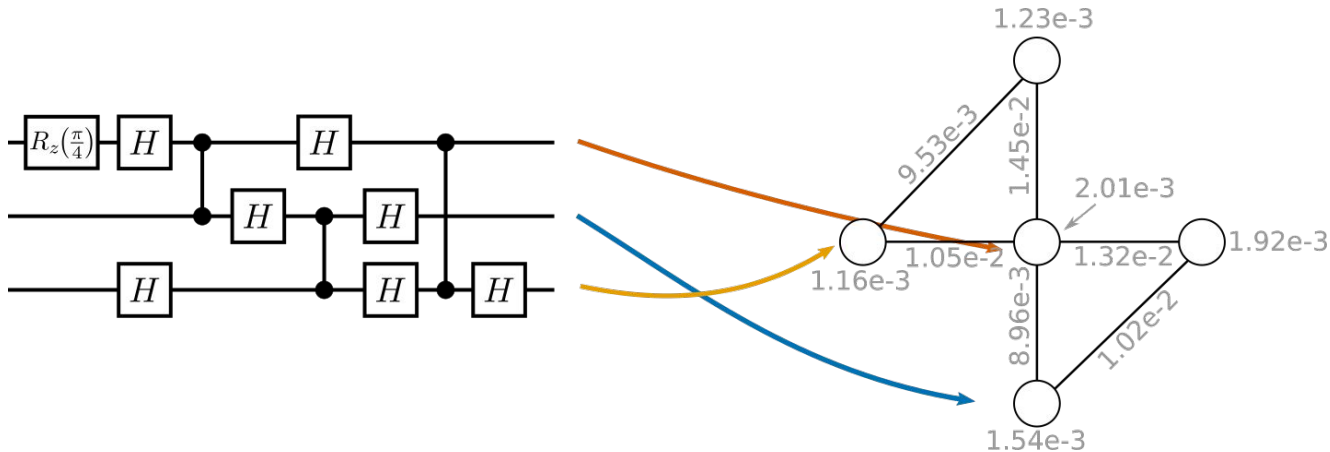
Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology





# Qubit placement and routing

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

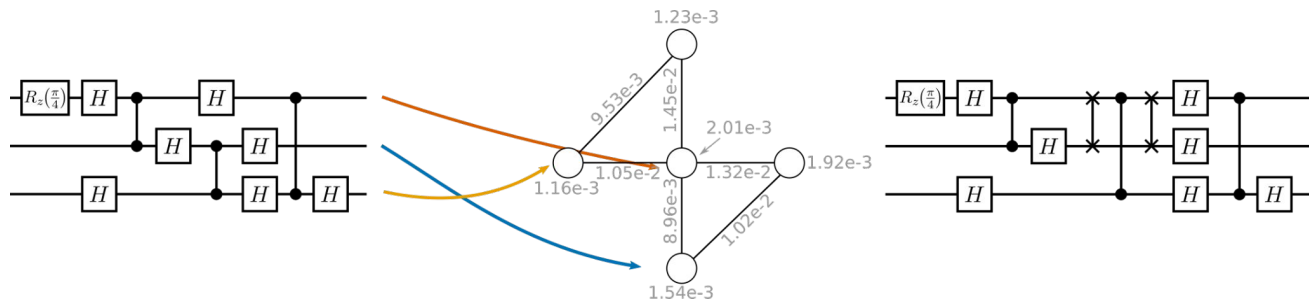
Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology



Placement is NP-complete; routing is NP hard.

# Qubit placement and routing

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology
- neutral atom machines can lose atoms

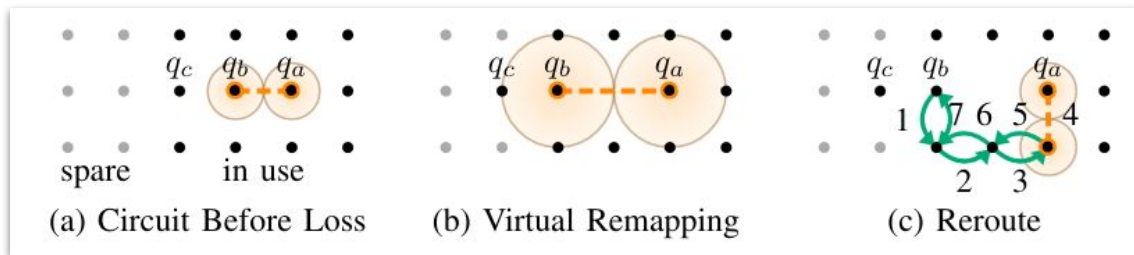


Image: Baker et al. (2021) *Exploiting Long-Distance Interactions and Tolerating Atom Loss in Neutral Atom Quantum Architectures*. ISCA '21.

# Qubit placement and routing

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Techniques can use optimal solvers, heuristics, or a combination of both.

Example: SABRE (“SWAP-based Bidirectional heuristic search)

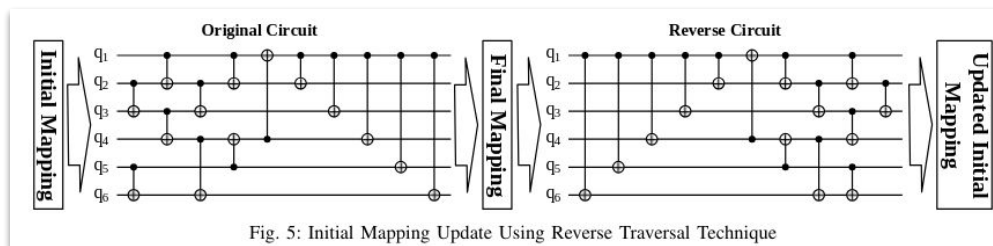


Fig. 5: Initial Mapping Update Using Reverse Traversal Technique

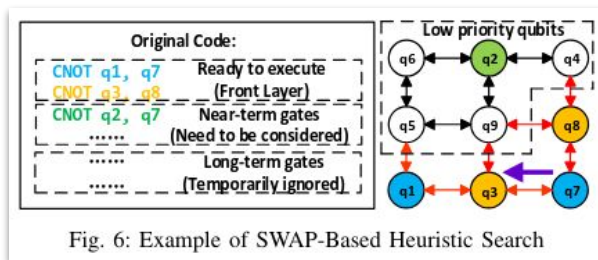
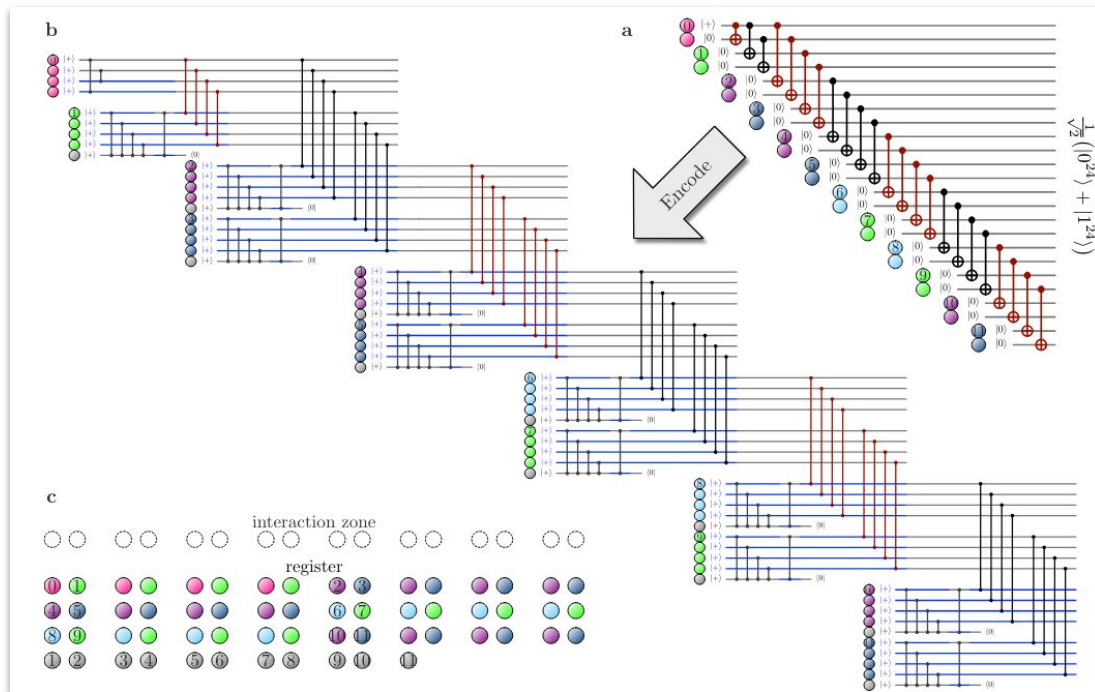


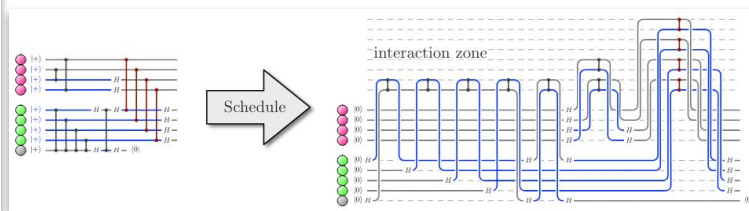
Fig. 6: Example of SWAP-Based Heuristic Search

# Gate scheduling



Recent demonstration of logical qubits on neutral atom machine: *move qubits in and out of "interaction zone" for 2-qubit gates*

(Microsoft + Atom computing + collaborators)



# Pulse design and optimization

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

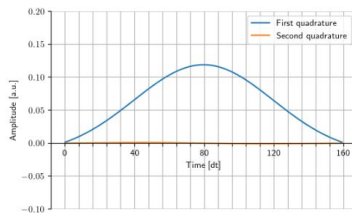
Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

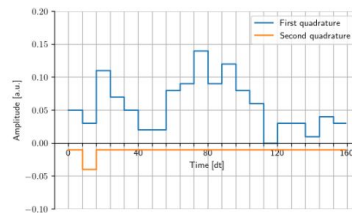
QPU

Electromagnetic pulses are what *actually* implement the gates in a gate. Interesting problems here:

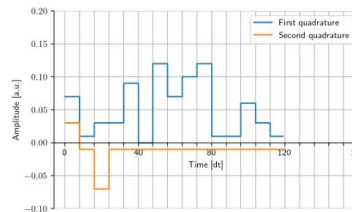
- optimizing duration/shape



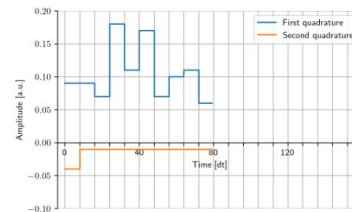
(a) DRAG  $X$  gate.



(b) Optimized  $X$  gate.



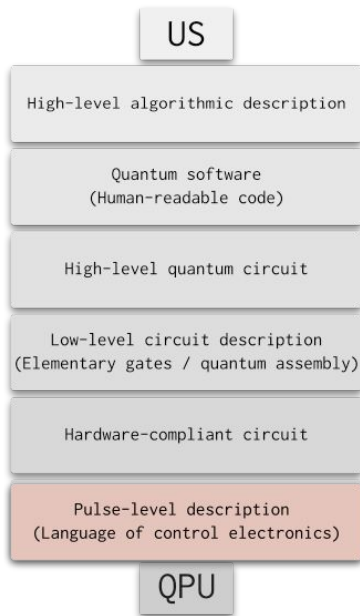
(c) Optimized  $X$  gate  $1.33\times$  faster.



(d) Optimized  $X$  gate  $2\times$  faster.

Image: E. Wright and R. de Sousa (2023) *Fast quantum gate design with deep reinforcement Learning using real-time feedback on readout signals*. arXiv:2305.01169 [quant-ph]

# Pulse design and optimization



Electromagnetic pulses are what *actually* implement the gates in a gate. Interesting problems here:

- optimizing duration/shape
- scheduling constraints

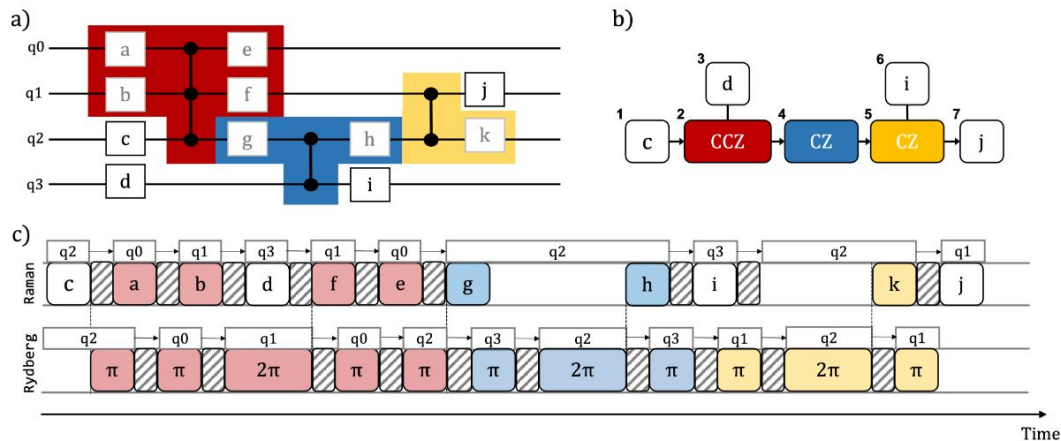


Image: R. B.-S. Tsai, H. Silv rio, L. Henri t (2022) *Pulse-Level Scheduling of Quantum Circuits for Neutral-Atom Devices*. Phys. Rev. Applied 18 064035.

## Software tools

Many general-purpose packages contain preset passes, and the ability to create your own.

```
qiskit_circuit = QuantumCircuit(...)

transpiled_circuit = transpile(
    qiskit_circuit,
    optimization_level=3,
    coupling_map=...,
    layout_method="sabre"
)
```

```
@qml.qnode(dev)
@expand_rot_and_remove_zeros
@qml.compile(pipeline=[
    qml.transforms.cancel_inverses,
    qml.transforms.single_qubit_fusion(),
    qml.transforms.commute_controlled(direction="left"),
    qml.transforms.merge_rotations()
], num_passes=3)
def tapered_circuit_simplified(params):
    for wire in dev.wires:
        if hf_state_tapered[wire] == 1:
            qml.PauliX(wires=wire)
```

See also: Cirq, TKET, staq, XACC, and more.

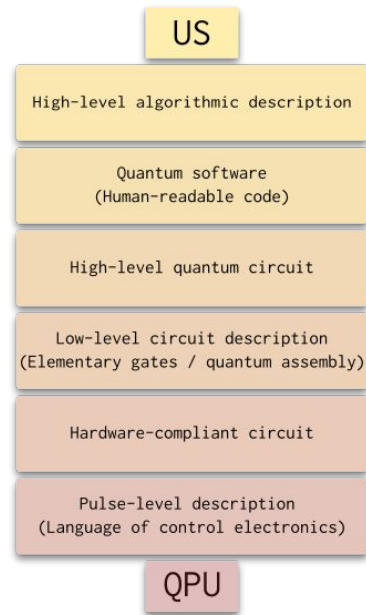
# Challenge

~~Problem:~~ many things are (computationally) hard

Many parts of the stack **still require expert input** and tailored design.

Need **automation** to scale up to (multi-QPU) devices w/1000s of qubits:

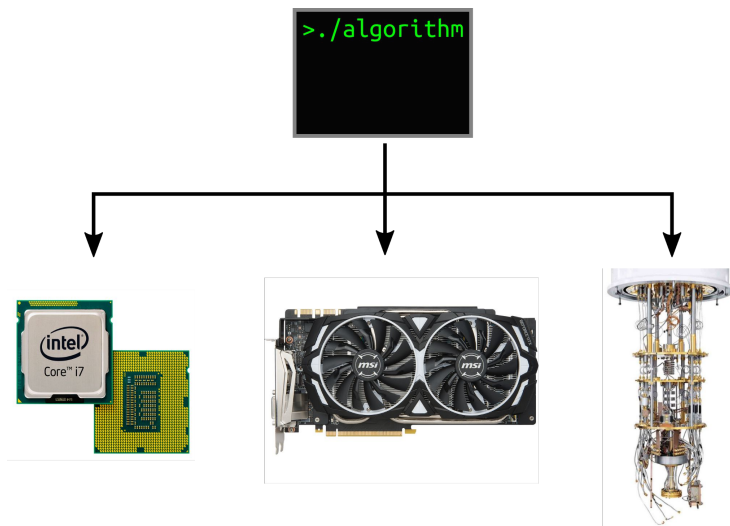
- circuit synthesis
- circuit optimization
- mapping, routing, and scheduling
- noise-aware techniques
- verification and debugging



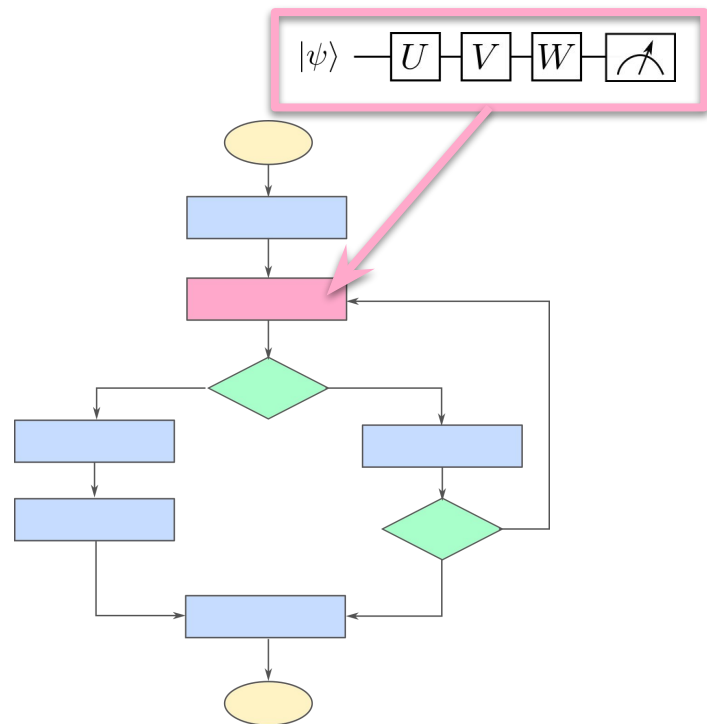


# Compiling classical and quantum code

Quantum algorithms don't exist in a vacuum.

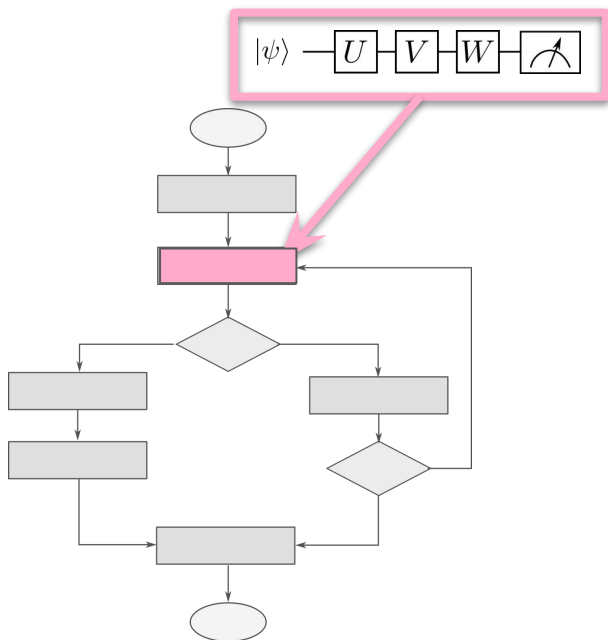


<https://phys.org/news/2012-04-intel-batch-ivy-bridge-processors.html>  
<https://www.techpowerup.com/img/17-03-28/65ee0f227ed0.jpg>  
(Photo/Justin Fantl, Rigetti Computing)

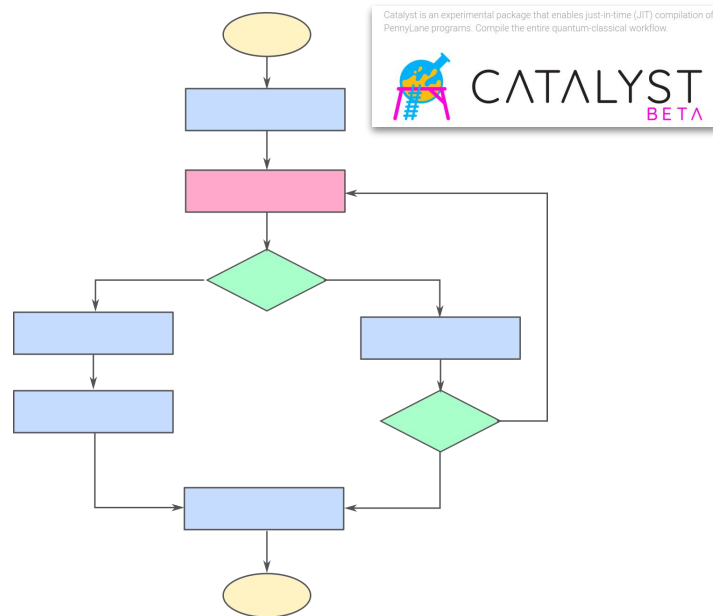


# Compiling classical and quantum code

So far, we've addressed the problem of compiling this part:



*Can we compile the whole thing?*



# Next time

## Content:

- Module 3: Quantum Fourier transform; towards Shor's algorithm

## Action items:

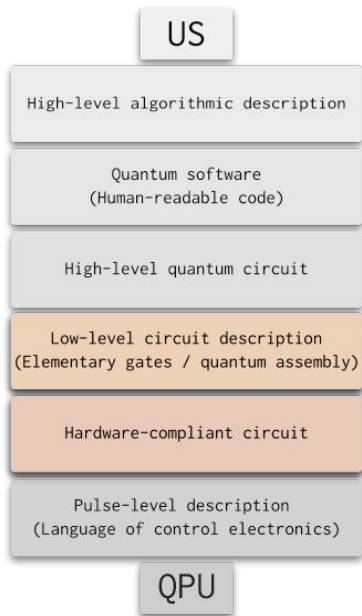
- Choose paper and post project group details to Piazza

## Recommended reading for next class:

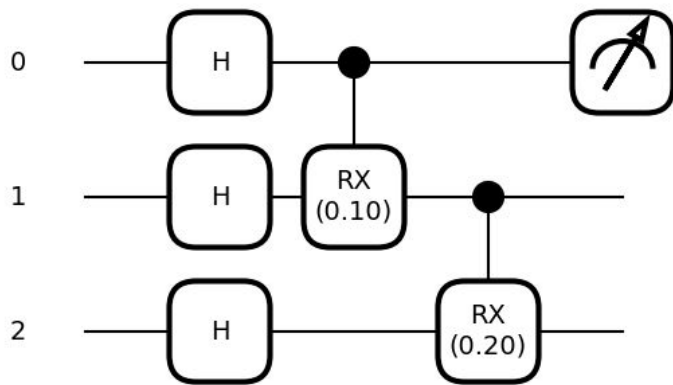
- Codebook QFT
- Nielsen and Chuang Ch. 5.1

```
def shors_algorithm(N):  
    p, q = 0, 0  
  
    while p * q != N:  
        a = np.random.choice(list(range(2, N - 1)))  
  
        if np.gcd(a, N) != 1:  
            p = np.gcd(a, N)  
            q = N // p  
            return p, q  
  
        sample = get_sample(a, N)  
        phase = fractional_binary_to_float(sample)  
        candidate_order = phase_to_order(phase, N)  
  
        if candidate_order % 2 == 0:  
            square_root = (a ** (candidate_order // 2)) % N  
  
            if square_root not in [1, N - 1]:  
                p = np.gcd(square_root - 1, N)  
                q = np.gcd(square_root + 1, N)  
  
    return p, q
```

# End-to-end example



Compile and optimize this circuit so it can run on a trapped-ion processor.



# End-to-end example

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

The native gate set used by some trapped-ion processors (e.g., IonQ) is

$$GPI(\phi) = \begin{pmatrix} 0 & e^{-i\phi} \\ e^{i\phi} & 0 \end{pmatrix}$$

$$GPI2(\phi) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -ie^{-i\phi} \\ -ie^{i\phi} & 1 \end{pmatrix}$$

$$MS = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 0 & -i \\ 0 & 1 & -i & 0 \\ 0 & -i & 1 & 0 \\ -i & 0 & 0 & 1 \end{pmatrix}$$

\*MS is actually a parametrized gate in general, but only this fixed-parameter version is implemented in hardware.

# End-to-end example

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

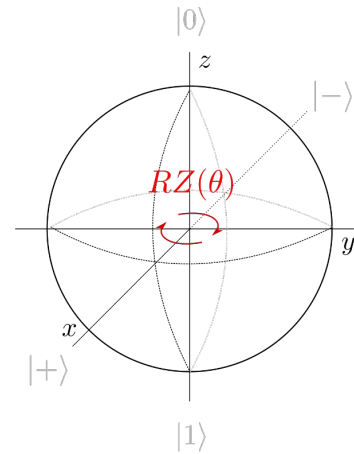
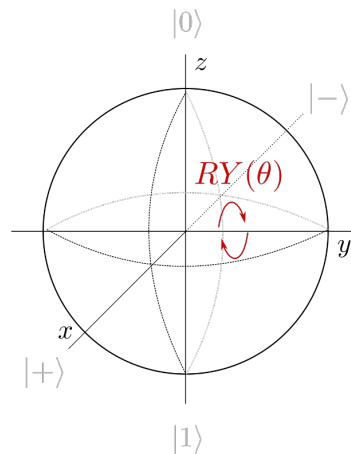
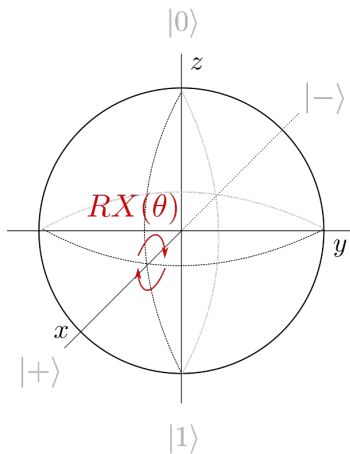
Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

$RX/RY/RZ$ : arbitrary-angle rotations about fixed axes.

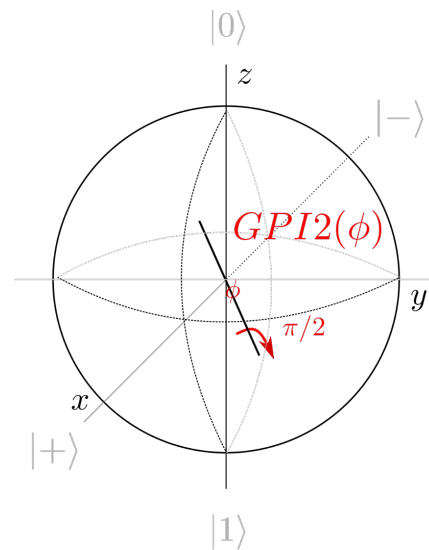
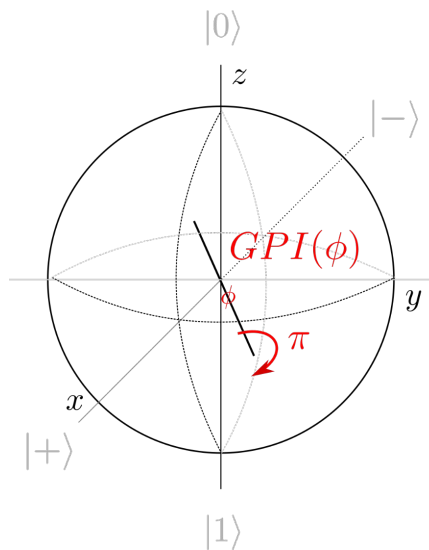
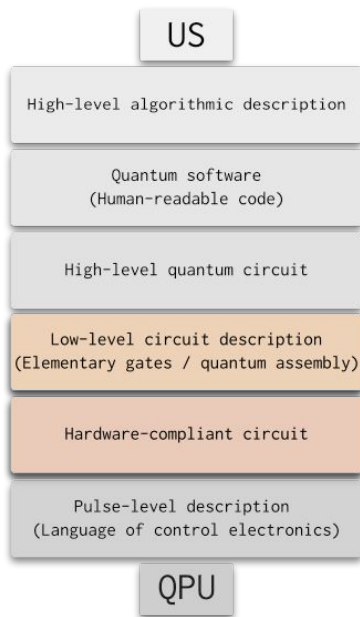


# End-to-end example

$$GPI(\phi) = \begin{pmatrix} 0 & e^{-i\phi} \\ e^{i\phi} & 0 \end{pmatrix}$$

$$GPI2(\phi) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -ie^{-i\phi} \\ -ie^{i\phi} & 1 \end{pmatrix}$$

*GPI/GPI2*: fixed-angle rotations about arbitrary axes in xy-plane.



# End-to-end example

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

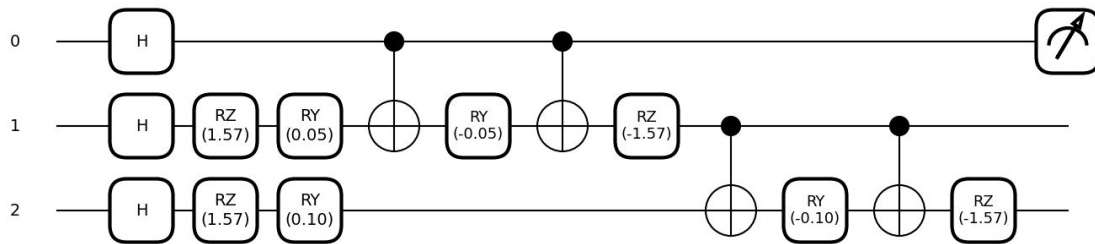
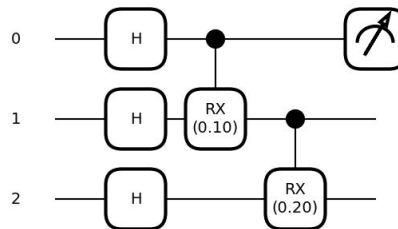
Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Unroll using known decompositions.





# End-to-end example

US

Map to native trapped-ion gates.

High-level algorithmic description

Quantum software  
(Human-readable code)

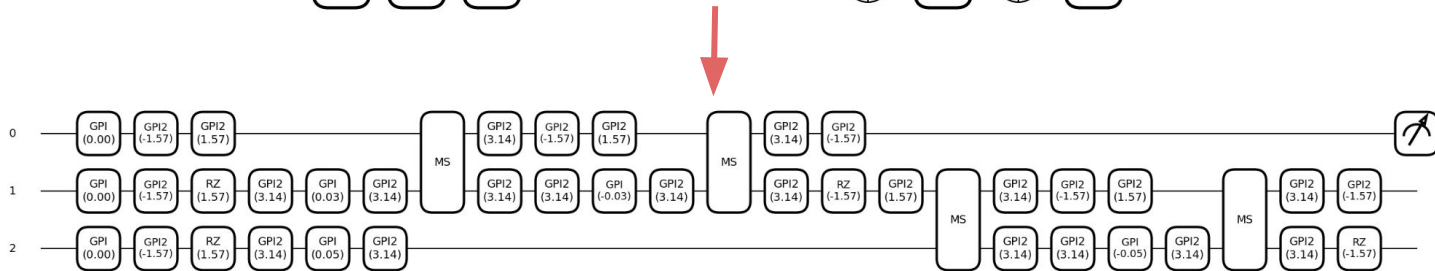
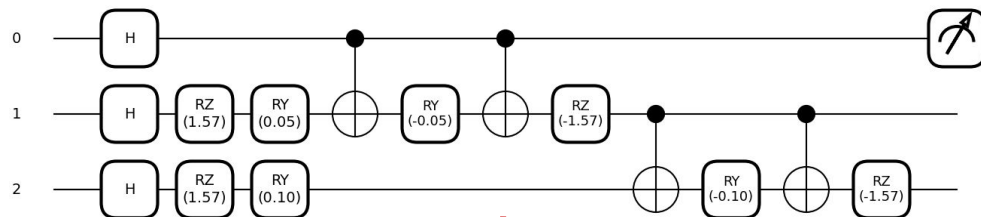
High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU



# End-to-end example

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

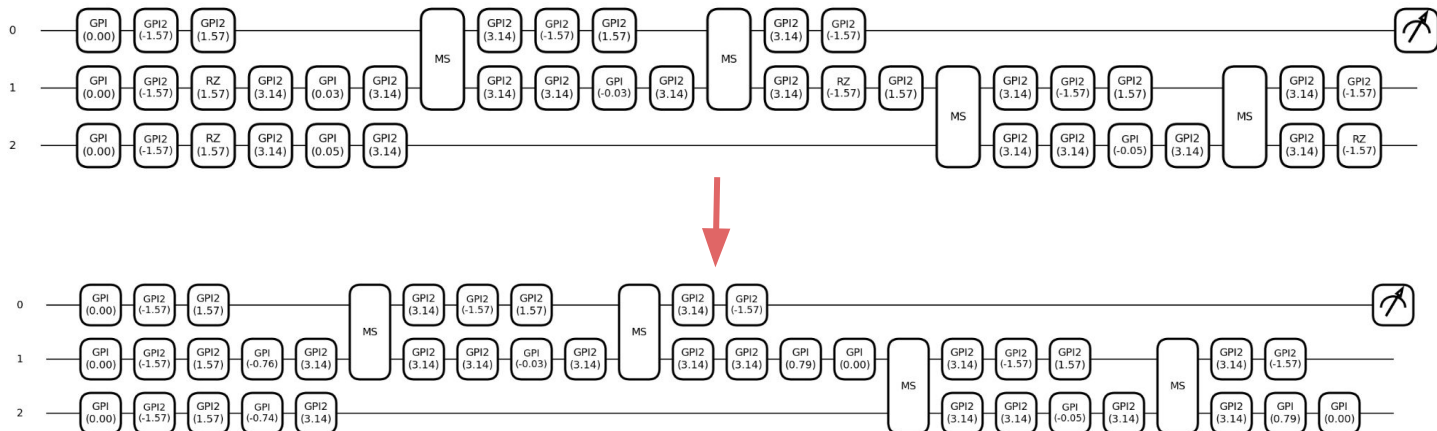
Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Hardware-specific optimization: virtually apply RZs



# End-to-end example

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

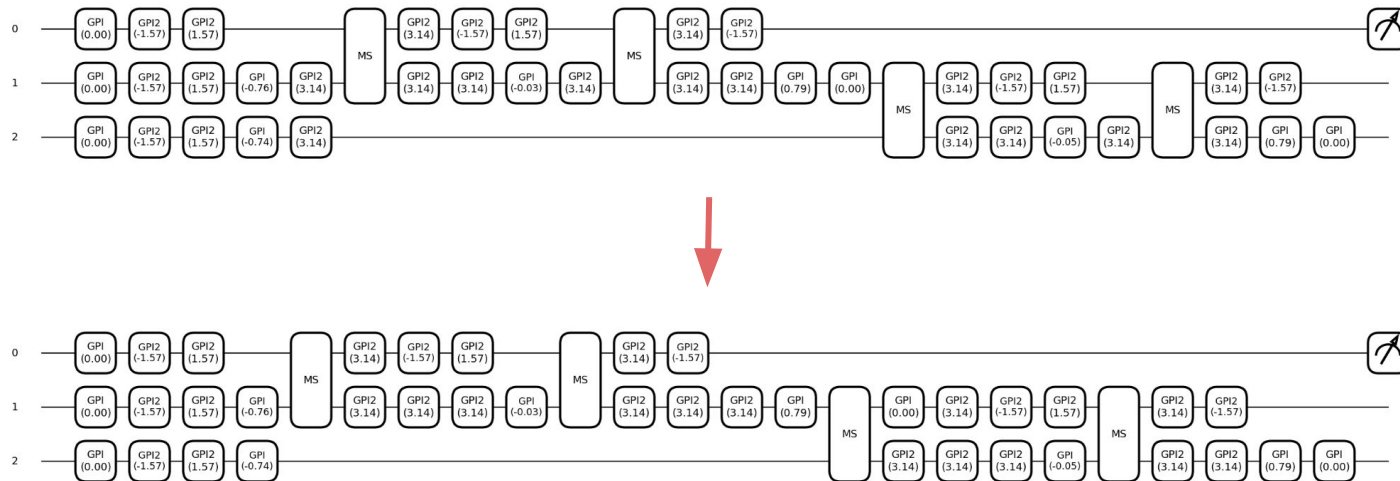
Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Optimization: exchange order of commuting gates



# End-to-end example

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

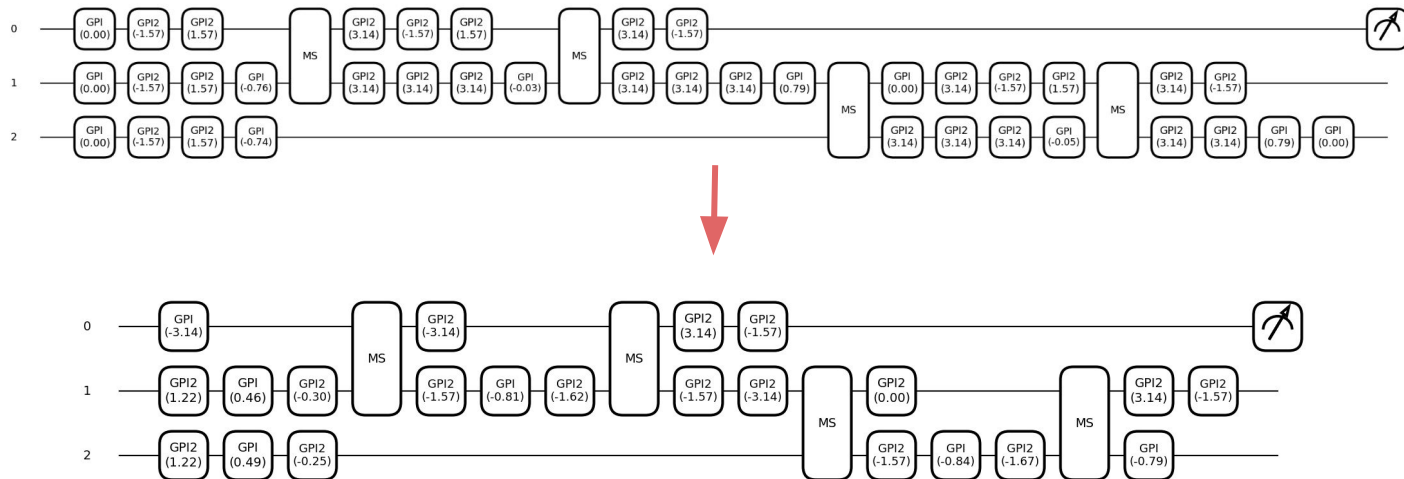
Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

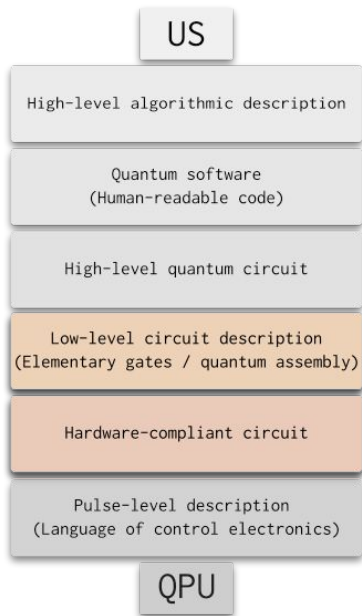
Pulse-level description  
(Language of control electronics)

QPU

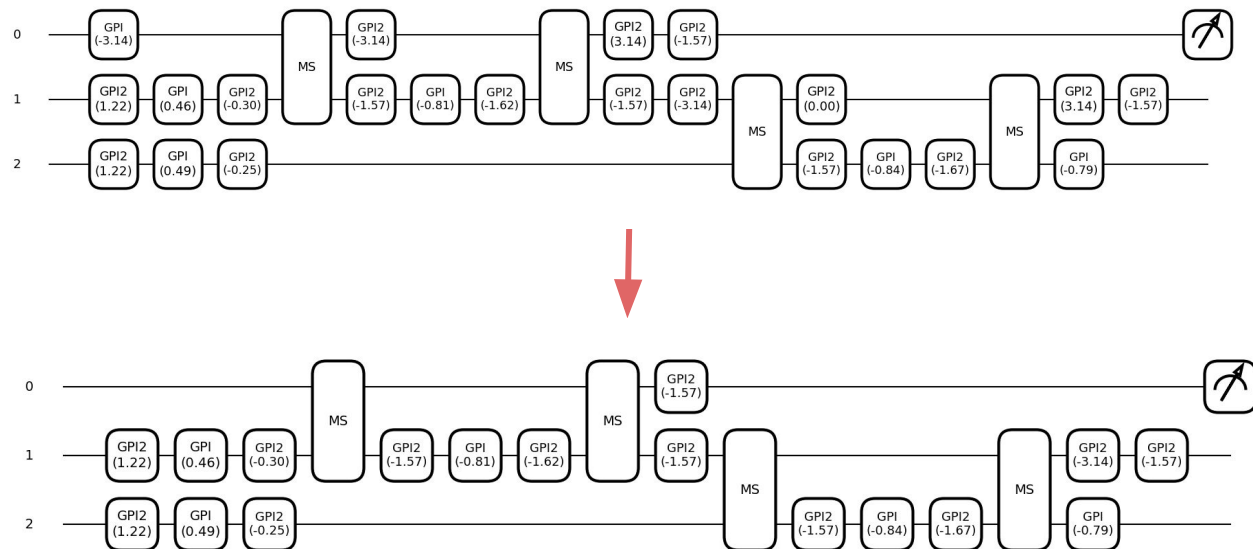
Optimization: fuse sequences of >3 single-qubit gates and apply circuit identities.



# End-to-end example



Repeat pushing/fusing/identity application.



# End-to-end example

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

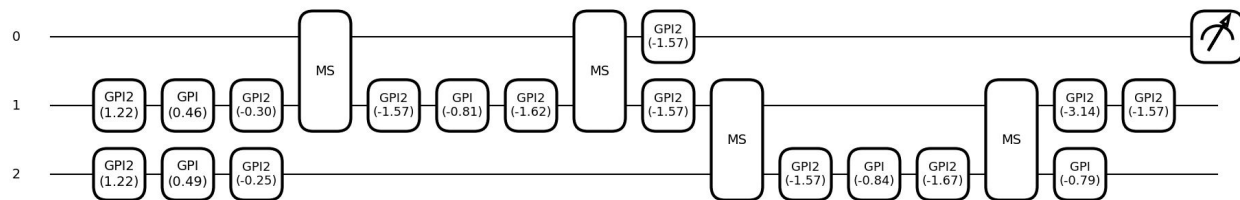
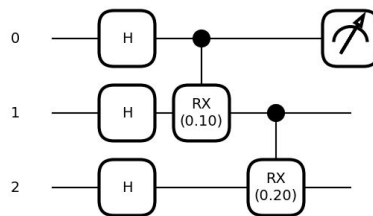
Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Final result



Q: How to choose the order of the passes?